Chapter 16

Case Study

This chapter is a systems performance case study: the story of a real-world performance issue, from initial report to final resolution. This particular issue occurred in a production cloud computing environment; I chose it as a routine example of systems performance analysis.

My intent in this chapter is not to introduce new technical content but to use storytelling to show how tools and methodologies may be applied in practice, in a real work environment. This should be especially useful for beginners who have yet to work on real-world systems performance issues, providing an over-the-shoulder view of how an expert approaches them, a commentary on what that expert might be thinking during the analysis, and why. This isn't necessarily documenting the best approach possible, but rather why one approach was taken.

16.1 An Unexplained Win

A microservice at Netflix was tested on a new container-based platform and was found to reduce request latency by a factor of three to four. While the container platform has many benefits, such a large gain was unexpected! This sounded too good to be true, and I was asked to investigate and explain how it happened.

For analysis I used a variety of tools, including those based on counters, static configuration, PMCs, software events, and tracing. All of these tool types played a role and provided clues that fit together. As this made for a broad tour of systems performance analysis, I used it as the opening story for my USENIX LISA 2019 talk on Systems Performance [Gregg 19h] and included it here as a case study.

16.1.1 Problem Statement

By speaking to the service team, I learned details of the microservice: It was a Java application for calculating customer recommendations, and was currently running on virtual machine instances in the AWS EC2 cloud. The microservice was composed of two components, and one of them was being tested on a new Netflix container platform called Titus, also running on AWS EC2. This component had a request latency of three to four seconds on the VM instances, which became one second on containers: three to four times faster!

The problem was to explain this performance difference. If it was simply due to the container move, the microservice can expect a permanent 3-4x win by moving. If it was due to some other factor, it would be worth understanding what it was and if it will be permanent. Perhaps it can also be applied elsewhere and to a greater degree.

What immediately came to mind was the benefit of running one component of a workload in isolation: it would be able to use the entire CPU caches without contention from the other component, improving cache hit ratios and thus performance. Another guess would be bursting on the container platform, where a container can use idle CPU resources from other containers.

16.1.2 Analysis Strategy

As traffic is handled by a load balancer (AWS ELB), it was possible to split the traffic between the VM and containers so that I could log in to both at the same time. This is an ideal situation for comparative analysis: I could run the same analysis command on both at the same time of day (same traffic mix and load) and compare the output immediately.

In this case I had access to the container host, not just the container, which allowed me to use any analysis tool and provided the permission for those tools to make any syscall. If I had had only container access, analysis would have been much more time-consuming due to limited observability sources and kernel permissions, requiring much more inference from limited metrics rather than direct measurement. Some performance issues are currently impractical to analyze from the container alone (see Chapter 11, Cloud Computing).

For methodologies, I planned to start with the 60-second checklist (Chapter 1, Introduction, Section 1.10.1, Linux Perf Analysis in 60s) and the USE method (Chapter 2, Methodologies, Section 2.5.9, The USE Method), and based on their clues perform drill-down analysis (Section 2.5.12, Drill-Down Analysis) and other methodologies.

I've included the commands I ran and their output in the following sections, using a "serverA#" prompt for the VM instance, and "serverB#" for the container host.

16.1.3 Statistics

I began by running uptime(1) to check the load average statistics. On both systems:

```
serverA# uptime
 22:07:23 up 15 days,  5:01,  1 user,  load average: 85.09, 89.25, 91.26

serverB# uptime
 22:06:24 up 91 days, 23:52,  1 user,  load average: 17.94, 16.92, 16.62
```

This showed that the load was roughly steady, getting a little lighter on the VM instance (85.09 compared to 91.26) and a little heavier on the container (17.94 compared to 16.62). I checked the trends to see whether the problem was increasing, decreasing, or steady: this is especially

important in cloud environments that can automatically migrate load away from an unhealthy instance. More than once I've logged in to a problematic instance to find little activity, and a one-minute load average approaching zero.

The load averages also showed that the VM had much higher load than the container host (85.09 versus 17.94), although I would need statistics from other tools to understand what this meant. High load averages usually point to CPU demand, but can be I/O-related as well (see Chapter 6, CPUs, Section 6.6.1, uptime).

To explore CPU load, I turned to mpstat(1), beginning with system-wide averages. On the virtual machine:

```
serverA# mpstat 10
Linux 4.4.0-130-generic (...) 07/18/2019        _x86_64_  (48 CPU)

10:07:55 PM  CPU   %usr  %nice  %sys %iowait  %irq %soft %steal %guest %gnice  %idle
10:08:05 PM  all  89.72   0.00  7.84    0.00  0.00  0.04   0.00   0.00   0.00   2.40
10:08:15 PM  all  88.60   0.00  9.18    0.00  0.00  0.05   0.00   0.00   0.00   2.17
10:08:25 PM  all  89.71   0.00  9.01    0.00  0.00  0.05   0.00   0.00   0.00   1.23
10:08:35 PM  all  89.55   0.00  8.11    0.00  0.00  0.06   0.00   0.00   0.00   2.28
10:08:45 PM  all  89.87   0.00  8.21    0.00  0.00  0.05   0.00   0.00   0.00   1.86
^C
Average:     all  89.49   0.00  8.47    0.00  0.00  0.05   0.00   0.00   0.00   1.99
```

And the container:

```
serverB# mpstat 10
Linux 4.19.26 (...) 07/18/2019        _x86_64_  (64 CPU)

09:56:11 PM CPU    %usr  %nice  %sys %iowait  %irq %soft %steal %guest %gnice  %idle
09:56:21 PM  all  23.21   0.01  0.32    0.00  0.00  0.10   0.00   0.00   0.00  76.37
09:56:31 PM  all  20.21   0.00  0.38    0.00  0.00  0.08   0.00   0.00   0.00  79.33
09:56:41 PM  all  21.58   0.00  0.39    0.00  0.00  0.10   0.00   0.00   0.00  77.92
09:56:51 PM  all  21.57   0.01  0.39    0.02  0.00  0.09   0.00   0.00   0.00  77.93
09:57:01 PM  all  20.93   0.00  0.35    0.00  0.00  0.09   0.00   0.00   0.00  78.63
^C
Average:     all  21.50   0.00  0.36    0.00  0.00  0.09   0.00   0.00   0.00  78.04
```

mpstat(1) prints the number of CPUs as the first line. The output showed that the virtual machine had 48 CPUs, and the container host had 64. This helped me further interpret the load averages: if they were CPU-based, it would show that the VM instance was running well into CPU saturation, because the load averages were roughly double the CPU count, whereas the container host was under-utilized. The mpstat(1) metrics supported this hypothesis: the idle time on the VM was around 2%, whereas on the container host it was around 78%.

By examining the other mpstat(1) statistics, I identified other leads:

- CPU utilization (`%usr` + `%sys` + ...) showed that the VM was at 98% versus the container at 22%. These processors have two hyperthreads per CPU core, so crossing the 50% utilization mark typically means hyperthread core contention, degrading performance. The VM was well into this territory, whereas the container host might still be benefiting from only one busy hyperthread per core.
- The system time (`%sys`) on the VM was much higher: around 8% versus 0.38%. If the VM was running at CPU saturation, this extra `%sys` time might include kernel context switching code paths. Kernel tracing or profiling could confirm.

I continued the other commands on the 60-second checklist. vmstat(8) showed run queue lengths similar to the load averages, confirming that the load averages were CPU-based. iostat(1) showed little disk I/O, and sar(1) showed little network I/O. (Those outputs are not included here.) This confirmed that the VM was running at CPU saturation, causing runnable threads to wait their turn, whereas the container host was not. top(1) on the container host also showed that only one container was running.

These commands provided statistics for the USE method, which also identified the issue of CPU load.

Had I solved the issue? I'd found that the VM had a load average of 85 on a 48-CPU system, and that this load average was CPU-based. This meant that threads were waiting their turn roughly 77% of the time (85/48 – 1), and eliminating this time spent waiting would produce a roughly 4x speedup (1 / (1 – 0.77)). While this magnitude corresponded to the issue, I couldn't yet explain why the load average was higher: more analysis was necessary.

16.1.4 Configuration

Knowing that there was a CPU issue, I checked the configuration of the CPUs and their limits (static performance tuning: Sections 2.5.17 and 6.5.7). The processors themselves were different between the VMs and containers. Here is /proc/cpuinfo for the virtual machine:

```
serverA# cat /proc/cpuinfo
processor       : 47
vendor_id       : GenuineIntel
cpu family      : 6
model           : 85
model name      : Intel(R) Xeon(R) Platinum 8175M CPU @ 2.50GHz
stepping        : 4
microcode       : 0x200005e
cpu MHz         : 2499.998
cache size      : 33792 KB
physical id     : 0
siblings        : 48
core id         : 23
cpu cores       : 24
apicid          : 47
```

```
initial apicid  : 47
fpu             : yes
fpu_exception   : yes
cpuid level     : 13
wp              : yes
flags           : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat
pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtscp lm constant_tsc
arch_perfmon rep_good nopl xtopology nonstop_tsc aperfmperf eagerfpu pni pclmulqdq
monitor ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes
xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch invpcid_single kaiser
fsgsbase tsc_adjust bmi1 hle avx2 smep bmi2 erms invpcid rtm mpx avx512f rdseed adx
smap clflushopt clwb avx512cd xsaveopt xsavec xgetbv1 ida arat
bugs            : cpu_meltdown spectre_v1 spectre_v2 spec_store_bypass
bogomips        : 4999.99
clflush size    : 64
cache_alignment : 64
address sizes   : 46 bits physical, 48 bits virtual
power management:
```

And the container:

```
serverB# cat /proc/cpuinfo
processor       : 63
vendor_id       : GenuineIntel
cpu family      : 6
model           : 79
model name      : Intel(R) Xeon(R) CPU E5-2686 v4 @ 2.30GHz
stepping        : 1
microcode       : 0xb000033
cpu MHz         : 1200.601
cache size      : 46080 KB
physical id     : 1
siblings        : 32
core id         : 15
cpu cores       : 16
apicid          : 95
initial apicid  : 95
fpu             : yes
fpu_exception   : yes
cpuid level     : 13
wp              : yes
flags           : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat
pse36 clflush mmx fxsr sse sse2 ht syscall nx pdpe1gb rdtscp lm constant_tsc arch_
perfmon rep_good nopl xtopology nonstop_tsc cpuid aperfmperf pni pclmulqdq monitor
est ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes
```

```
xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch cpuid_fault
invpcid_single pti fsgsbase bmi1 hle avx2 smep bmi2 erms invpcid rtm rdseed adx
xsaveopt ida
bugs            : cpu_meltdown spectre_v1 spectre_v2 spec_store_bypass l1tf
bogomips        : 4662.22
clflush size    : 64
cache_alignment : 64
address sizes   : 46 bits physical, 48 bits virtual
power management:
```

The CPUs for the container host had a slightly slower base frequency (2.30 versus 2.50 GHz); however, they had a much larger last-level cache (45 versus 33 Mbytes). Depending on the workload, the larger cache size can make a significant difference to CPU performance. To investigate further, I needed to use PMCs.

16.1.5 PMCs

Performance monitoring counters (PMCs) can explain CPU cycle performance, and are available on certain instances in AWS EC2. I've published a toolkit for PMC analysis on the cloud [Gregg 20e], which includes pmcarch(8) (Section 6.6.11, pmcarch). pmcarch(8) shows the Intel "architectural set" of PMCs, which are the most basic set commonly available.

On the virtual machine:

```
serverA# ./pmcarch -p 4093 10
K_CYCLES  K_INSTR    IPC BR_RETIRED   BR_MISPRED  BMR% LLCREF      LLCMISS     LLC%
982412660 575706336 0.59 126424862460 2416880487  1.91 15724006692 10872315070 30.86
999621309 555043627 0.56 120449284756 2317302514  1.92 15378257714 11121882510 27.68
991146940 558145849 0.56 126350181501 2530383860  2.00 15965082710 11464682655 28.19
996314688 562276830 0.56 122215605985 2348638980  1.92 15558286345 10835594199 30.35
979890037 560268707 0.57 125609807909 2386085660  1.90 15828820588 11038597030 30.26
[...]
```

On the container instance:

```
serverB# ./pmcarch -p 1928219 10
K_CYCLES  K_INSTR    IPC BR_RETIRED   BR_MISPRED  BMR% LLCREF      LLCMISS     LLC%
147523816 222396364 1.51 46053921119  641813770   1.39 8880477235  968809014  89.09
156634810 229801807 1.47 48236123575  653064504   1.35 9186609260  1183858023 87.11
152783226 237001219 1.55 49344315621  692819230   1.40 9314992450  879494418  90.56
140787179 213570329 1.52 44518363978  631588112   1.42 8675999448  712318917  91.79
136822760 219706637 1.61 45129020910  651436401   1.44 8689831639  617678747  92.89
[...]
```

This showed instructions per cycle (IPC) of around 0.57 for the VM versus around 1.52 for the container: a 2.6x difference.

One reason for the lower IPC could be hyperthread contention, as the VM host was running at over 50% CPU utilization. The last column showed an additional reason: the last-level cache (LLC) hit ratio was only 30% for the VM, versus around 90% for the container. This would cause instructions on the VM to frequently stall on main memory access, driving down IPC and instruction throughput (performance).

The lower LLC hit ratio on the VM could be due to at least three factors:

- A smaller LLC size (33 versus 45 Mbytes).
- Running the full workload instead of a subcomponent (as mentioned in the Problem Statement); a subcomponent will likely cache better: fewer instructions and data.
- CPU saturation causing more context switching, and jumping between code paths (including user and kernel), increasing cache pressure.

The last factor could be investigated using tracing tools.

16.1.6 Software Events

To investigate context switches, I began with the perf(1) command to count the system-wide context switch rate. This uses a software event, which is similar to a hardware event (PMC) but implemented in software (see Chapter 4, Observability Tools, Figure 4.5, and Chapter 13, perf, Section 13.5, Software Events).

On the virtual machine:

```
serverA# perf stat -e cs -a -I 1000
#           time             counts unit events
     1.000411740          2,063,105      cs
     2.000977435          2,065,354      cs
     3.001537756          1,527,297      cs
     4.002028407            515,509      cs
     5.002538455          2,447,126      cs
     6.003114251          2,021,182      cs
     7.003665091          2,329,157      cs
     8.004093520          1,740,898      cs
     9.004533912          1,235,641      cs
    10.005106500          2,340,443      cs
^C    10.513632795          1,496,555      cs
```

This output showed a rate of around two million context switches per second. I then ran it on the container host, this time matching on the PID of the container application to exclude other possible containers (I did similar PID matching on the VM, and it did not noticeably change the previous results[1]):

[1] Then why don't I include the PID-matching output for the VM? I don't have it.

```
serverB# perf stat -e cs -p 1928219 -I 1000
#           time             counts unit events
     1.001931945              1,172      cs
     2.002664012              1,370      cs
     3.003441563              1,034      cs
     4.004140394              1,207      cs
     5.004947675              1,053      cs
     6.005605844                955      cs
     7.006311221                619      cs
     8.007082057              1,050      cs
     9.007716475              1,215      cs
    10.008415042              1,373      cs
^C    10.584617028                894      cs
```

This output showed a rate of only about one thousand context switches per second.

A high rate of context switches can put more pressure on the CPU caches, which are switching between different code paths, including the kernel code to manage the context switch, and possibly different processes.[2] To investigate context switches further, I used tracing tools.

16.1.7 Tracing

There are several BPF-based tracing tools for analyzing CPU usage and context switching further, including, from BCC: cpudist(8), cpuwalk(8), runqlen(8), runqlat(8), runqslower(8), cpuunclaimed(8), and more (see Figure 15.1).

cpudist(8) shows the on-CPU duration of threads. On the virtual machine:

```
serverA# cpudist -p 4093 10 1
Tracing on-CPU time... Hit Ctrl-C to end.

     usecs               : count     distribution
         0 -> 1          : 3618650  |****************************************|
         2 -> 3          : 2704935  |*****************************           |
         4 -> 7          : 421179   |****                                    |
         8 -> 15         : 99416    |*                                       |
        16 -> 31         : 16951    |                                        |
        32 -> 63         : 6355     |                                        |
        64 -> 127        : 3586     |                                        |
       128 -> 255        : 3400     |                                        |
       256 -> 511        : 4004     |                                        |
       512 -> 1023       : 4445     |                                        |
```

[2] For some processor and kernel configurations, context switching may also flush the L1 cache.

```
     1024 -> 2047       : 8173     |                                        |
     2048 -> 4095       : 9165     |                                        |
     4096 -> 8191       : 7194     |                                        |
     8192 -> 16383      : 11954    |                                        |
    16384 -> 32767      : 1426     |                                        |
    32768 -> 65535      : 967      |                                        |
    65536 -> 131071     : 338      |                                        |
   131072 -> 262143     : 93       |                                        |
   262144 -> 524287     : 28       |                                        |
   524288 -> 1048575    : 4        |                                        |
```

This output shows that the application typically spent very little time on-CPU, often less than 7 microseconds. Other tools (stackcount(8) of t:sched:sched_switch, and /proc/PID/status) showed that the application was usually leaving the CPU due to involuntary[3] context switches.

On the container host:

```
serverB# cpudist -p 1928219 10 1
Tracing on-CPU time... Hit Ctrl-C to end.

     usecs               : count     distribution
         0 -> 1          : 0        |                                        |
         2 -> 3          : 16       |                                        |
         4 -> 7          : 6        |                                        |
         8 -> 15         : 7        |                                        |
        16 -> 31         : 8        |                                        |
        32 -> 63         : 10       |                                        |
        64 -> 127        : 18       |                                        |
       128 -> 255        : 40       |                                        |
       256 -> 511        : 44       |                                        |
       512 -> 1023       : 156      |*                                       |
      1024 -> 2047       : 238      |**                                      |
      2048 -> 4095       : 4511     |****************************************|
      4096 -> 8191       : 277      |**                                      |
      8192 -> 16383      : 286      |**                                      |
     16384 -> 32767      : 77       |                                        |
     32768 -> 65535      : 63       |                                        |
     65536 -> 131071     : 44       |                                        |
    131072 -> 262143     : 9        |                                        |
    262144 -> 524287     : 14       |                                        |
    524288 -> 1048575    : 5        |                                        |
```

[3] /proc/PID/status calls them `nonvoluntary_ctxt_switches`.

Now the application was typically spending between 2 and 4 milliseconds on-CPU. Other tools showed that it was not interrupted much by involuntary context switches.

The involuntary context switches on the VM, and subsequent high rate of context switches seen earlier, caused performance problems. Causing the application to leave the CPU after often less than 10 microseconds also does not give the CPU caches much time to warm up to the current code paths.

16.1.8 Conclusion

I concluded that the reasons for the performance gain were:

- **No container neighbors**: The container host was idle except for the one container. This allowed the container to have the entire CPU caches all to itself, as well as to run without CPU contention. While this produced container-favorable results during the test, it is not the expected situation for long-term production use where neighboring containers will be the norm. The microservice may find that the 3-4x performance win vanishes when other tenants move in.
- **LLC size and workload difference**: The IPC was 2.6 times lower on the VM, which could explain 2.6x of this slowdown. One cause was likely hyperthread contention as the VM host was running at over 50% utilization (and had two hyperthreads per core). However, the main cause was likely the lower LLC hit ratio: 30% on the VM versus 90% on the container. This low LLC hit ratio had three probable reasons:
  - A smaller LLC size on the VM: 33 Mbytes versus 45 Mbytes.
  - A more complex workload on the VM: the full app, requiring more instruction text and data, versus the component run on the container.
  - A high rate of context switches on the VM: around 2 million per second. These prevent threads from running on-CPU for long, interfering with cache warmup. The on-CPU durations were typically less than 10 μs on the VM compared to 2-4 ms on the container host.
- **CPU load difference**: A higher load was directed to the VM, driving the CPUs to saturation: a CPU-based load average of 85 on a 48-CPU system. This caused a rate of around 2 million context switches per second, and run queue latency as threads waited their turn. The run queue latency implied by the load averages showed the VM was running roughly 4x slower.

These issues explain the observed performance difference.

16.2 Additional Information

For more case studies in systems performance analysis, check the bug database (or ticketing system) at your company for previous performance-related issues, and the public bug databases for the applications and operating system you use. These issues often begin with a problem statement and finish with the final fix. Many bug database systems also include a timestamped comments history, which can be studied to see the progression of analysis, including hypotheses

explored and wrong turns taken. Taking wrong turns, and identifying multiple contributing factors, is normal.

Some systems performance case studies are published from time to time, for example, as on my blog [Gregg 20j]. Technical journals with a focus on practice, such as *USENIX ;login:* [USENIX 20] and *ACM Queue* [ACM 20], also often use case studies as context when describing new technical solutions to problems.

16.3 References

[Gregg 19h] Gregg, B., "LISA2019 Linux Systems Performance," *USENIX LISA*, http://www.brendangregg.com/blog/2020-03-08/lisa2019-linux-systems-performance.html, 2019.

[ACM 20] "acmqueue," http://queue.acm.org, accessed 2020.

[Gregg 20e] Gregg, B., "PMC (Performance Monitoring Counter) Tools for the Cloud," https://github.com/brendangregg/pmc-cloud-tools, last updated 2020.

[Gregg 20j] "Brendan Gregg's Blog," http://www.brendangregg.com/blog, last updated 2020.

[USENIX 20] ";login: The USENIX Magazine," https://www.usenix.org/publications/login, accessed 2020.

This page intentionally left blank